



Universidad Nacional Autónoma de
México

Facultad de Ingeniería

Reporte S05: Mexi Banco con Docker Compose

Asignatura: Sistemas Distribuidos

Profesor: Oscar Manuel Ruiz Hurtado

Grupo: 02

Semestre: 2027-1

Alumno:

Arroyo Onofre Leonardo

Introducción

Docker Compose permite levantar varios servicios que pertenecen a una misma aplicación. En esta práctica se utilizó Mexi Banco junto con PostgreSQL. La API se probó con Postman para comprobar cuentas, transferencias, SPEI, movimientos, avisos y errores. El volumen de Docker se utilizó para conservar la información de la base de datos aun cuando los contenedores fueran eliminados.

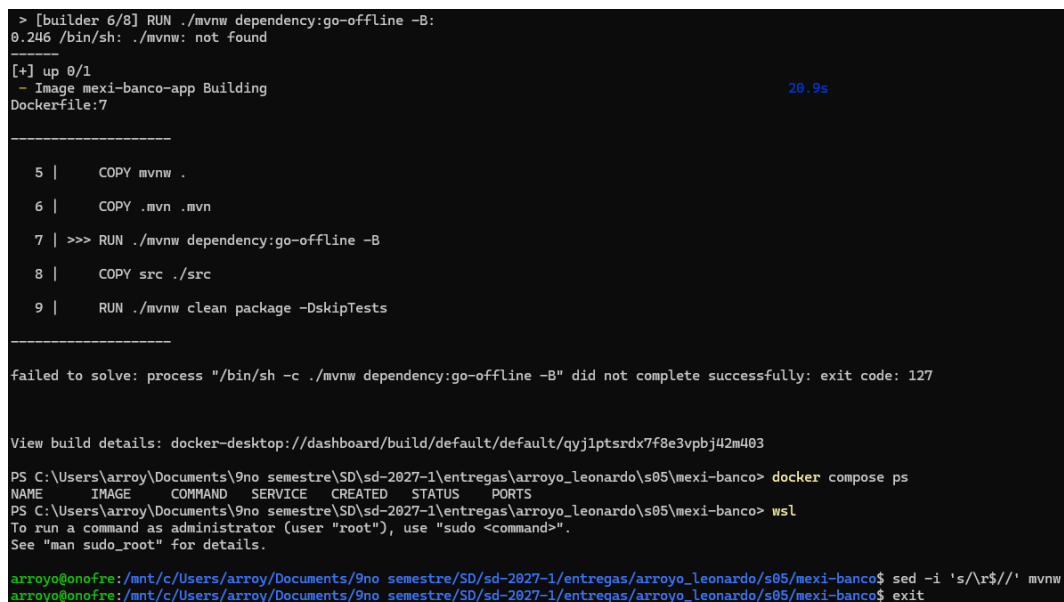
Desarrollo

Paso 1. Clonación del repositorio. Primero se clonó la rama v05.1 del repositorio y se entró al directorio del proyecto. De esta forma se obtuvo la versión indicada para realizar la práctica.

```
1 git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
2 cd mexi-banco
```

Paso 2. Levantamiento con Compose. En este paso se dio el primer intento de ejecutar `docker compose up -build -d`. No obstante, la construcción tuvo un fallo durante la ejecución de `./mvnw`: Docker indicó que el archivo no pudo ejecutar ciertos procedimientos debido a conflictos con los símbolos y finalizó con el código 127. El problema se relacionó con los saltos de línea de Windows que contenía el wrapper de Maven al ser ejecutado dentro de Linux.

Para solucionarlo, se entró al proyecto desde WSL y se ejecutó `sed -i 's/\r$//' mvnw`. Este comando eliminó el retorno de carro de cada línea de `mvnw`; después se repitió el levantamiento de Compose. **Paso 2.** En la captura se observa el primer intento, el mensaje `./mvnw: not found` y el comando utilizado desde WSL para corregir el archivo.



```
> [builder 6/8] RUN ./mvnw dependency:go-offline -B:
0.246 /bin/sh: ./mvnw: not found

[+] up 0/1
- Image mexi-banco-app Building
Dockerfile:7
20.9s

5 | COPY mvnw .
6 | COPY .mvn .mvn
7 | >>> RUN ./mvnw dependency:go-offline -B
8 | COPY src ./src
9 | RUN ./mvnw clean package -DskipTests

failed to solve: process "/bin/sh -c ./mvnw dependency:go-offline -B" did not complete successfully: exit code: 127

View build details: docker-desktop://dashboard/build/default/default/qyjlptsrdx7f8e3vpbj42m403

PS C:\Users\arroy\Documents\9no semestre\SD\sd-2027-1\entregas\arroyo_leonardo\s05\mexi-banco> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
PS C:\Users\arroy\Documents\9no semestre\SD\sd-2027-1\entregas\arroyo_leonardo\s05\mexi-banco> wsl
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

arroyo@onofre:/mnt/c/Users/arroy/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s05/mexi-banco$ sed -i 's/\r$//' mvnw
arroyo@onofre:/mnt/c/Users/arroy/Documents/9no semestre/SD/sd-2027-1/entregas/arroyo_leonardo/s05/mexi-banco$ exit
```

Figura 1: primer intento de ejecutar Docker Compose y corrección aplicada al archivo `mvnw`.

Una vez corregido el archivo, Compose construyó la imagen, creó la red y generó el volumen. Al consultar el estado de los servicios, `app` y `db` aparecieron como `healthy`.

Paso 2. Esta evidencia muestra que el segundo levantamiento finalizó correctamente y que la aplicación ya estaba lista para recibir peticiones.

```
#16 8.137 [INFO] --- spring-boot:3.3.4:repackage (repackage) @ mexi-banco ---
#16 8.617 [INFO] Replacing main artifact /app/target/mexi-banco-0.1.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
#16 8.617 [INFO] The original artifact has been renamed to /app/target/mexi-banco-0.1.0.jar.original
#16 8.618 [INFO] BUILD SUCCESS
#16 8.618 [INFO] -----
#16 8.618 [INFO] Total time: 7.835 s
#16 8.620 [INFO] Finished at: 2026-09-26T13:34:58Z
#16 8.620 [INFO] -----
#16 DONE 8.7s

#17 [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
#17 DONE 0.1s

#18 exporting to image
#18 exporting layers
#18 exporting layers 1.6s done
#18 exporting manifest sha256:c6375c3156678e2c0f6be2fba8a96687ce7a3aa9d387236742d5d0ec83d46e0a 0.0s done
#18 exporting config sha256:48718124eb6fc984d2849de146ec4653f76a28997483df0fa97cb09431a4d6b8e 0.0s done
#18 exporting attestation manifest sha256:348325fe4c3a6a56427d17a92588828f4735f778e8433b0f42c3e702fbb2df35 0.0s done
#18 exporting manifest list sha256:835d86748b1918bf29dd0fbd4ee83abcd568490715837e8e5ced4afe6f6053e 0.0s done
#18 naming to docker.io/library/mexi-banco-app:latest done
#18 unpacking to docker.io/library/mexi-banco-app:latest
#18 unpacking to docker.io/library/mexi-banco-app:latest 0.3s done
#18 DONE 2.1s

#19 resolving provenance for metadata file
#19 DONE 0.0s
[*] up 5/5
✔Image mexi-banco-app Built 92.5s
✔Network mexibanco Created 0.8s
✔Volume mexi-banco_mexibanco_datos Created 0.8s
✔Container mexi-banco-db-1 Healthy 7.6s
✔Container mexi-banco-app-1 Started 7.2s

PS C:\Users\arroyo\Documents\9no semestre\SD\sd-2027-1\entregas\arroyo_leonardo\s05\mexi-banco> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
mexi-banco-app-1    mexi-banco-app       "java -jar app.jar"    app        About a minute ago    Up 57 seconds (healthy)    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine   "docker-entrypoint.s..." db          About a minute ago    Up About a minute (healthy)    5432/tcp
```

Figura 2: construcción correcta de la imagen y verificación de los servicios saludables.

Paso 3. Pruebas de Postman. La colección se dividió en cinco grupos: disponibilidad, transferencia interna, idempotencia, consultas del monolito y manejo de errores.

Paso 3. En esta captura se observa el índice de la colección y el orden en que se enviaron las peticiones.

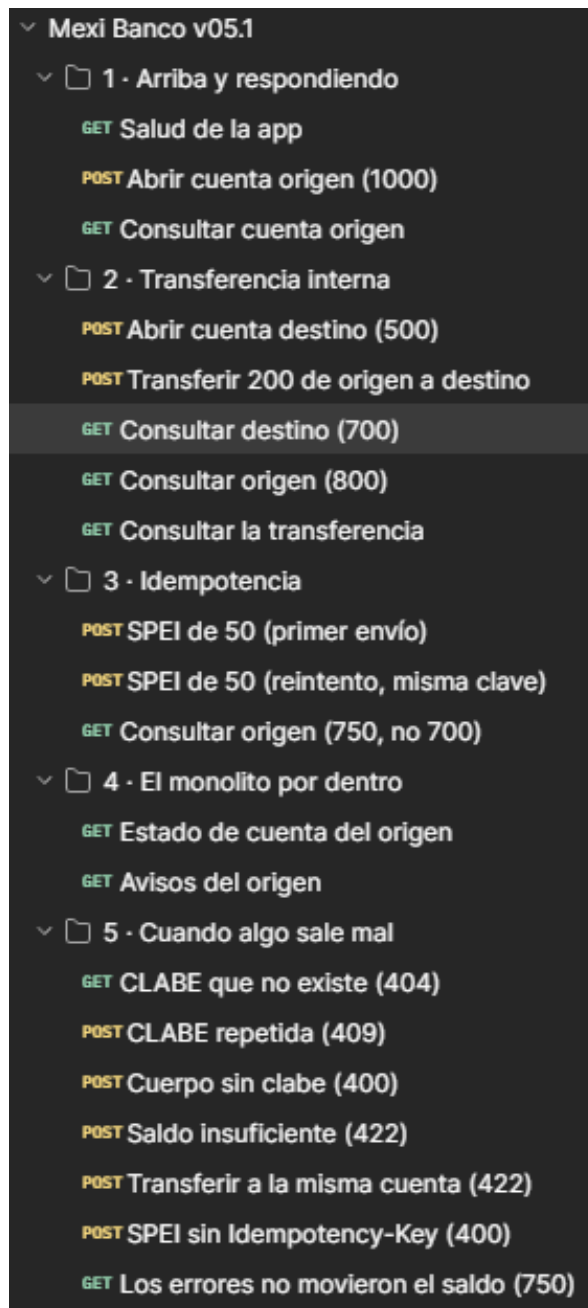


Figura 3: grupos de peticiones de la colección Mexi Banco v05.1 en Postman.

Paso 3.1. Arriba y respondiendo. Primero se envió una petición GET al endpoint de salud. La respuesta fue 200 OK y el estado fue UP, por lo que se comprobó que la aplicación estaba disponible.

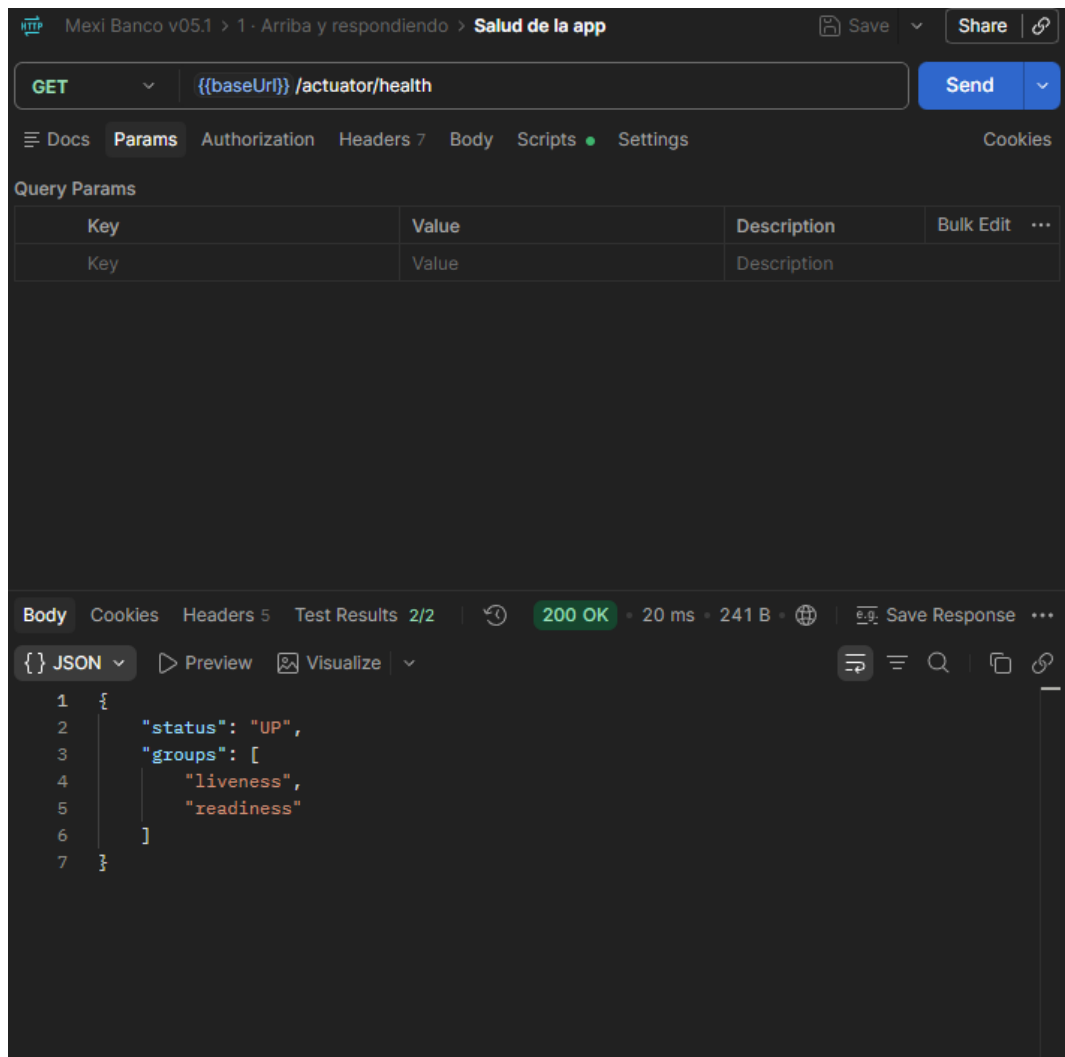


Figura 4: consulta de salud de la aplicación.

Paso 3.1. Arriba y respondiendo. Después se envió una petición POST para abrir la cuenta origen de Ana. La API respondió 201 **Created** y asignó un saldo inicial de 1000.

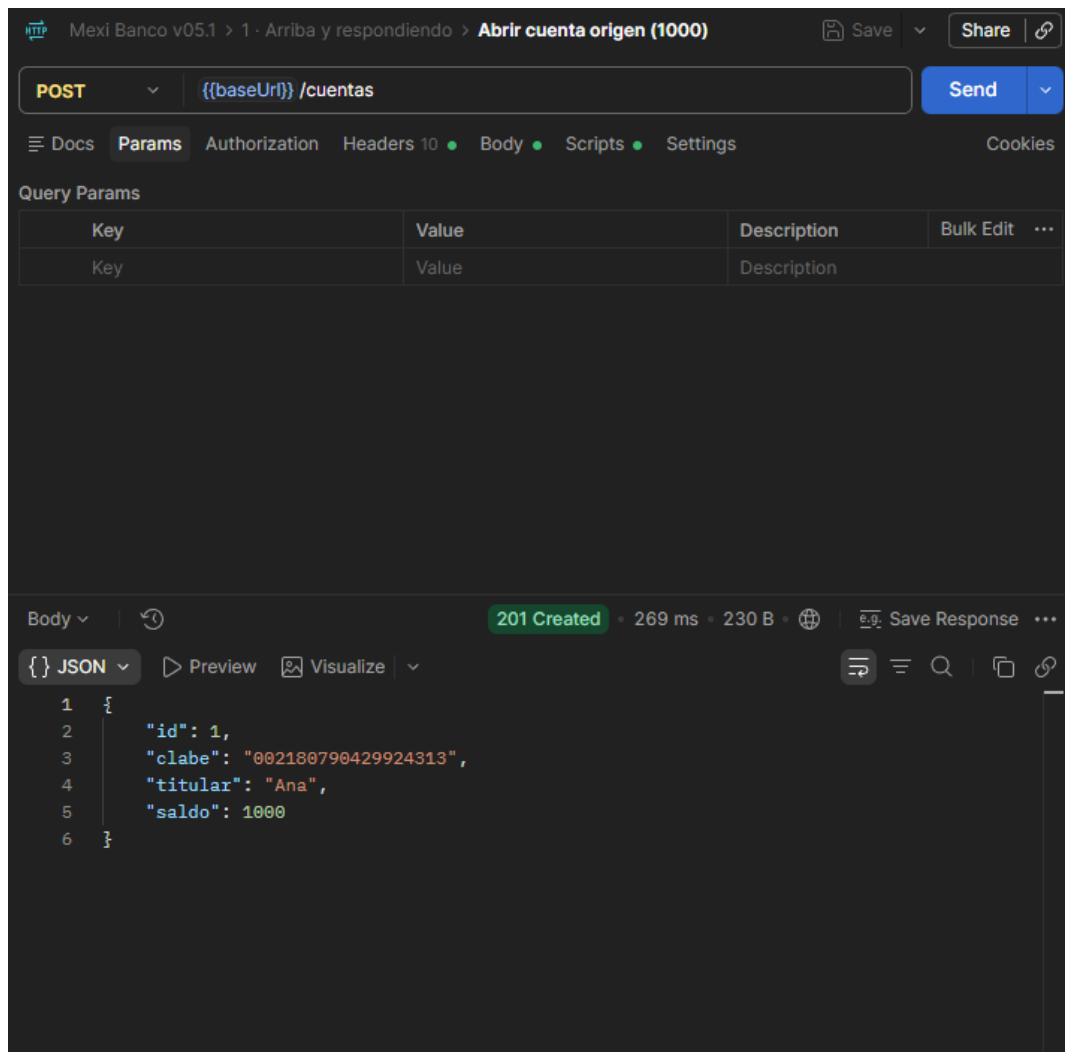


Figura 5: apertura de la cuenta origen con saldo inicial de 1000.

Paso 3.1. Arriba y respondiendo. Luego se consultó la cuenta origen con GET. La respuesta 200 OK mostró que la cuenta fue almacenada con el saldo creado anteriormente.

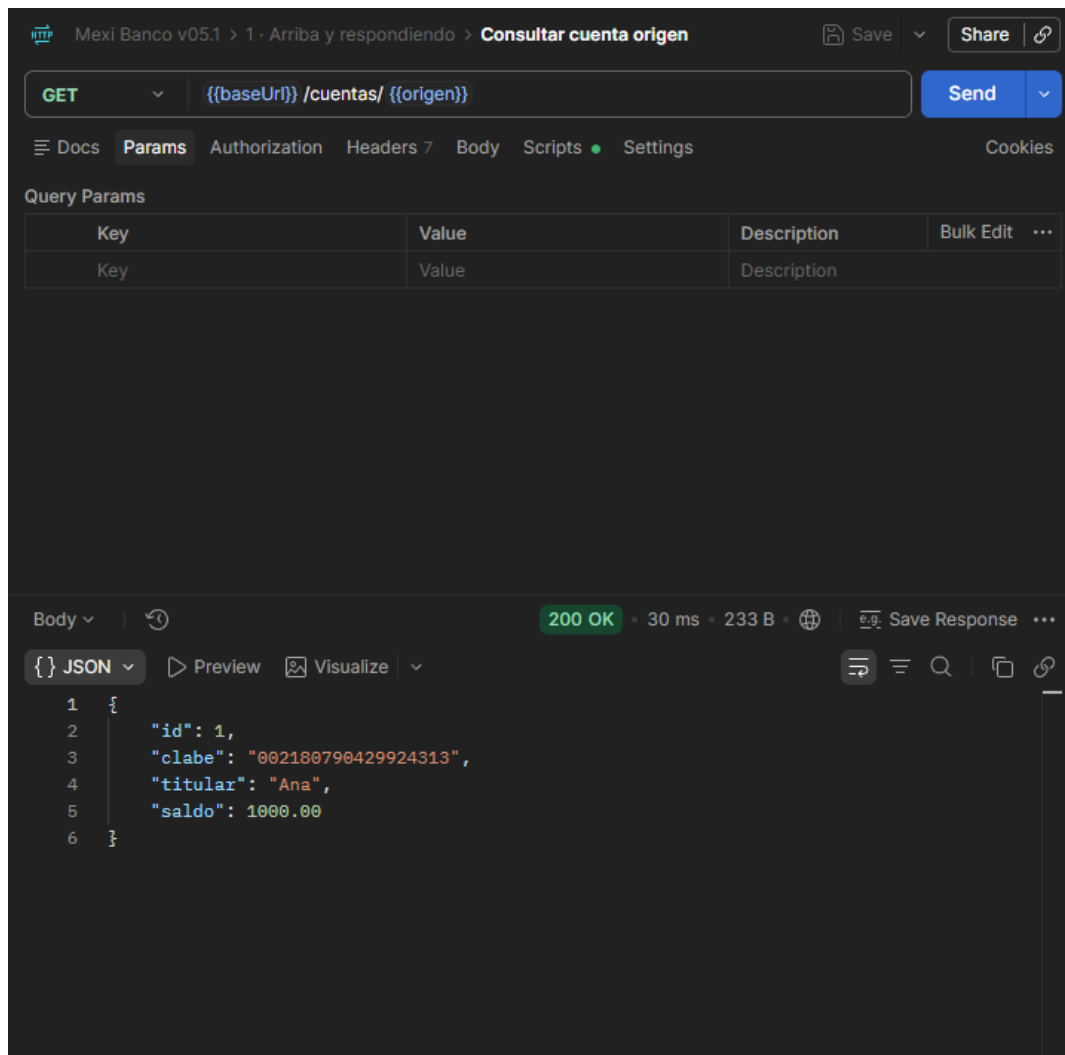


Figura 6: consulta de la cuenta origen recién creada.

Paso 3.2. Transferencia interna. Antes de transferir saldo se abrió la cuenta destino de Beto. La respuesta 201 Created mostró un saldo inicial de 500.

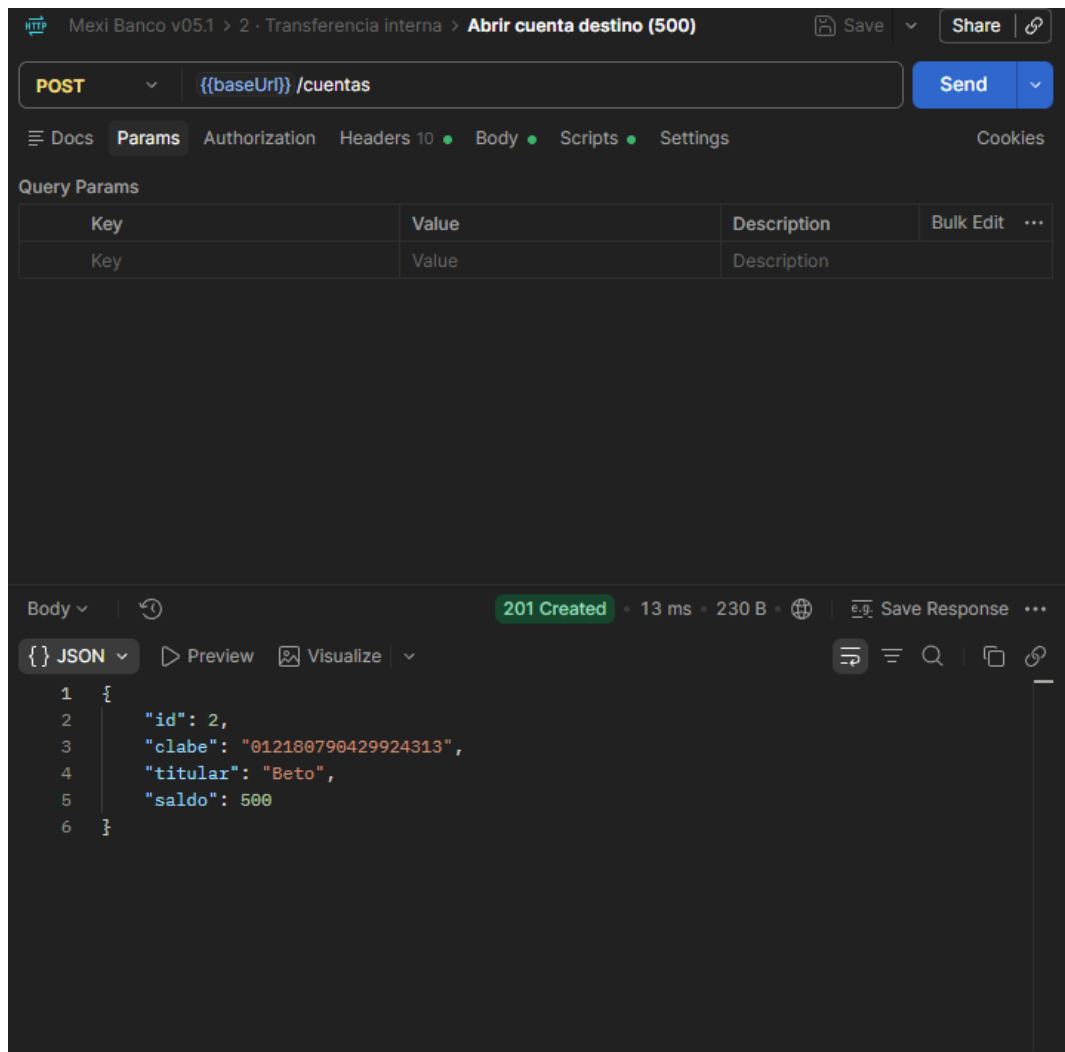


Figura 7: apertura de la cuenta destino con saldo inicial de 500.

Paso 3.2. Transferencia interna. Se envió una petición POST de transferencia por 200 desde Ana hacia Beto. La respuesta 201 Created confirmó el registro de la operación.

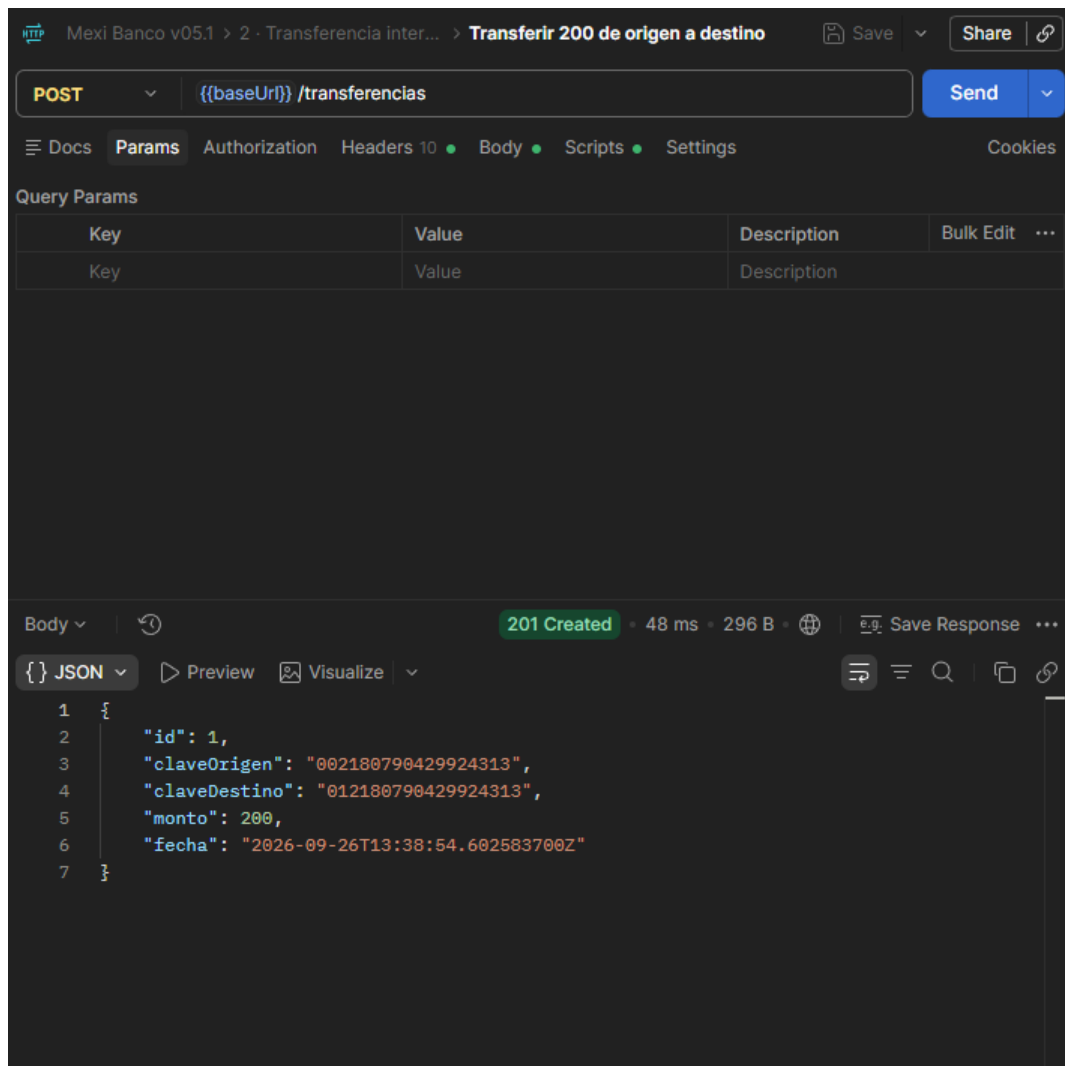


Figura 8: transferencia interna de 200 desde la cuenta origen hacia la destino.

Paso 3.2. Transferencia interna. Al consultar la cuenta destino, el saldo fue 700. Esto confirmó que la transferencia sumó los 200 al saldo inicial de Beto.

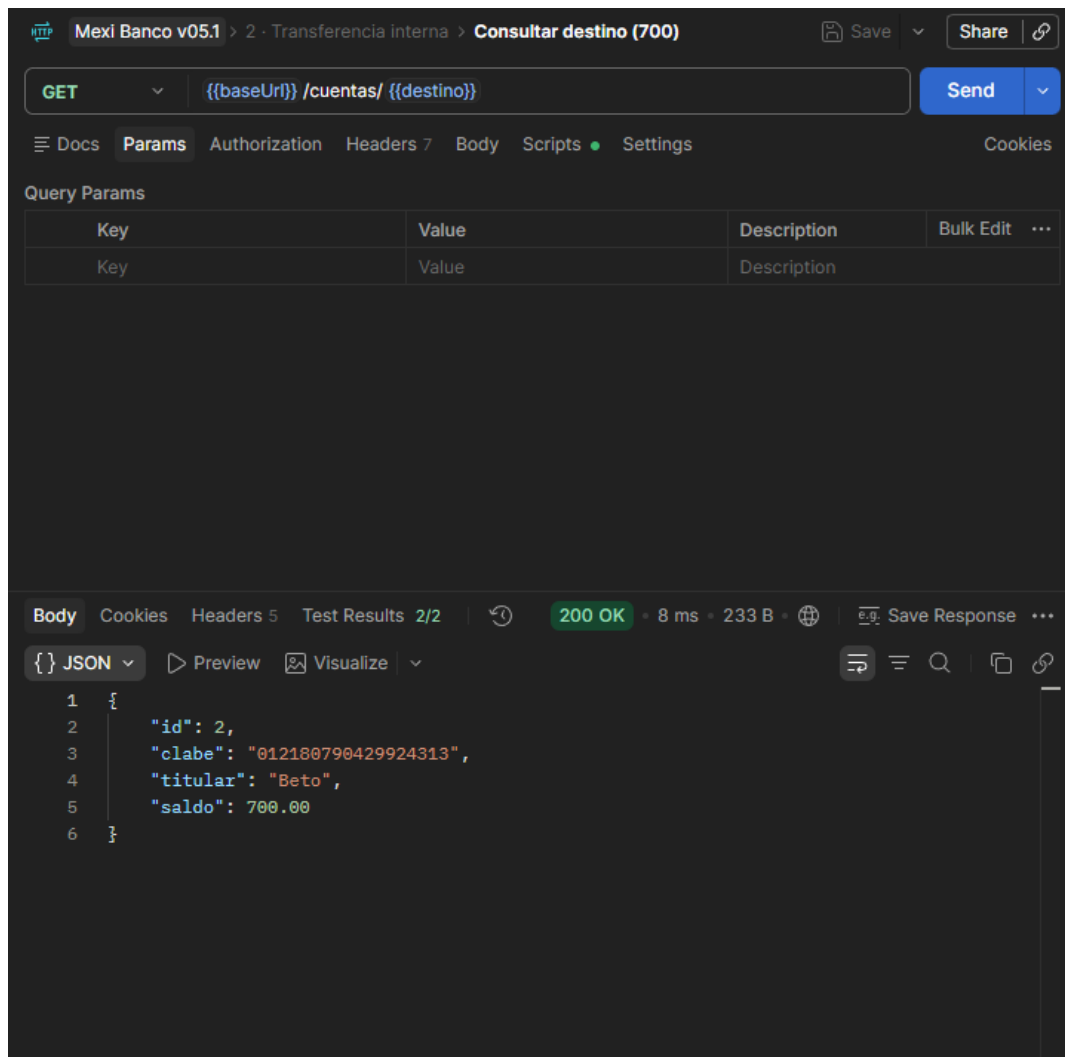


Figura 9: consulta de la cuenta destino después de recibir la transferencia.

Paso 3.2. Transferencia interna. Al consultar la cuenta origen, el saldo fue 800. Con ello se verificó que el mismo monto se descontó de Ana.

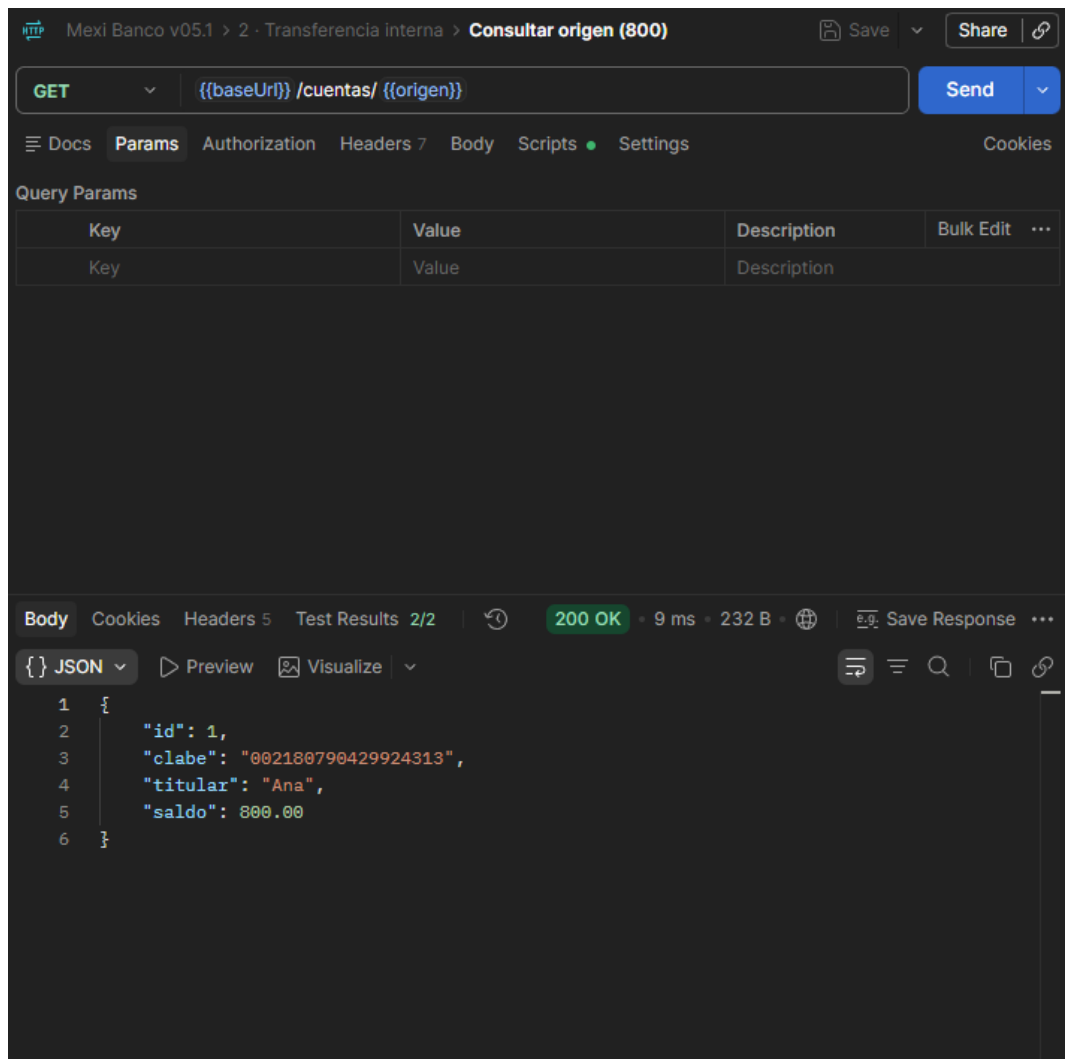


Figura 10: consulta de la cuenta origen después de enviar la transferencia.

Paso 3.2. Transferencia interna. Finalmente se consultó la transferencia mediante su identificador. La API devolvió los datos de origen, destino, monto y fecha de la operación realizada.

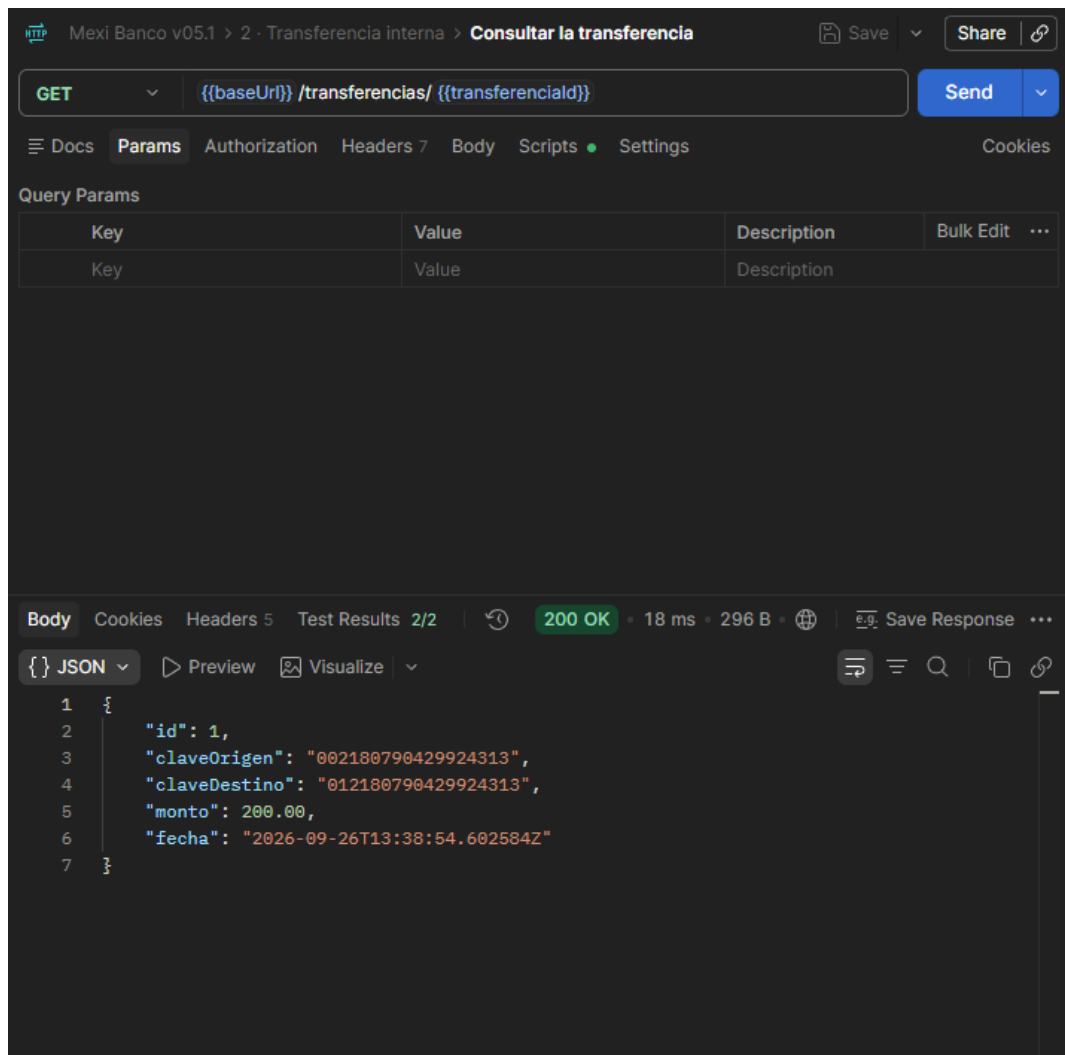


Figura 11: consulta del registro de la transferencia realizada.

Paso 3.3. Idempotencia. Se envió un SPEI por 50 utilizando una clave de idempotencia. La respuesta 201 Created mostró que el envío se registró con el estado ENVIADO.

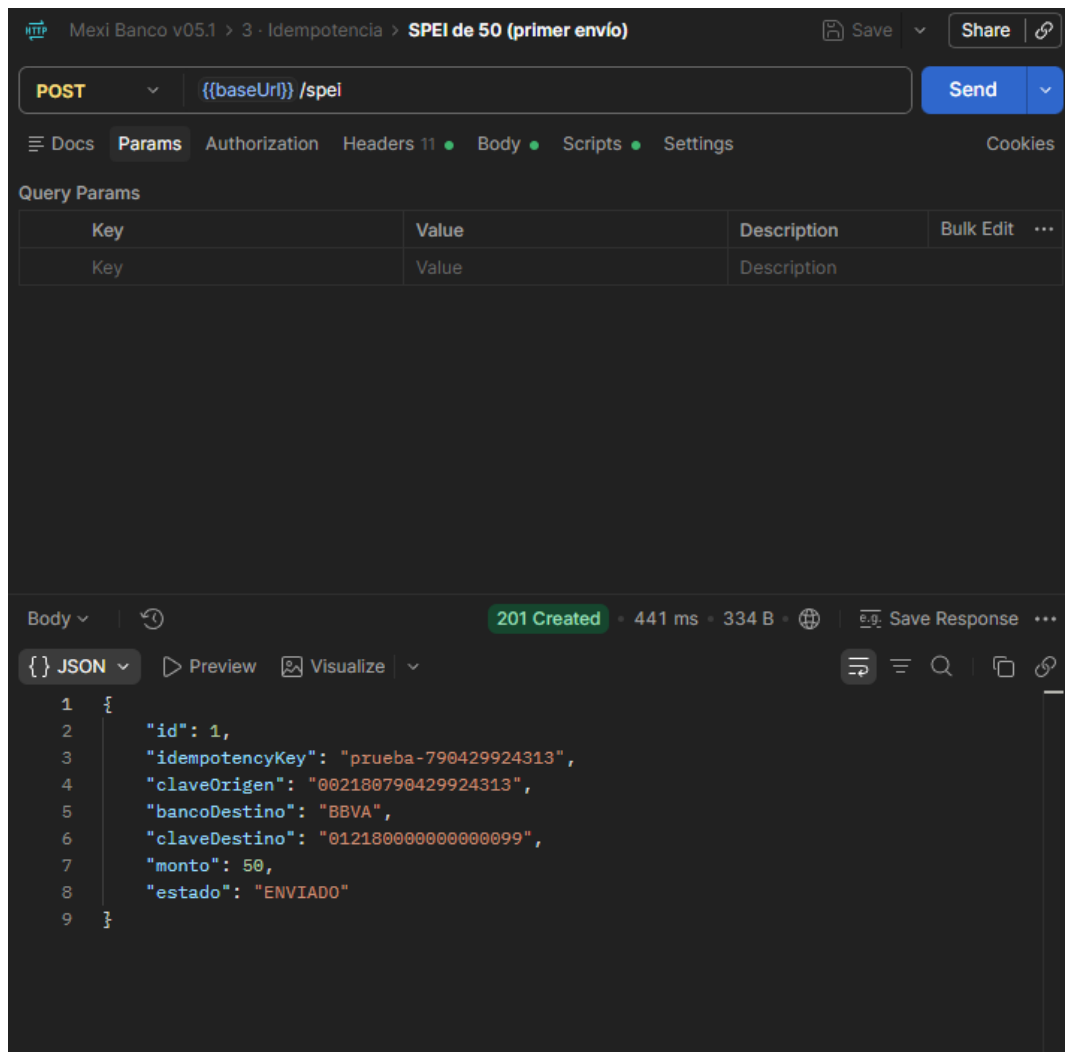


Figura 12: primer envío de SPEI por 50.

Paso 3.3. Idempotencia. Luego se repitió la misma petición con la misma clave. La respuesta devolvió el mismo identificador, por lo que no se creó un segundo SPEI.

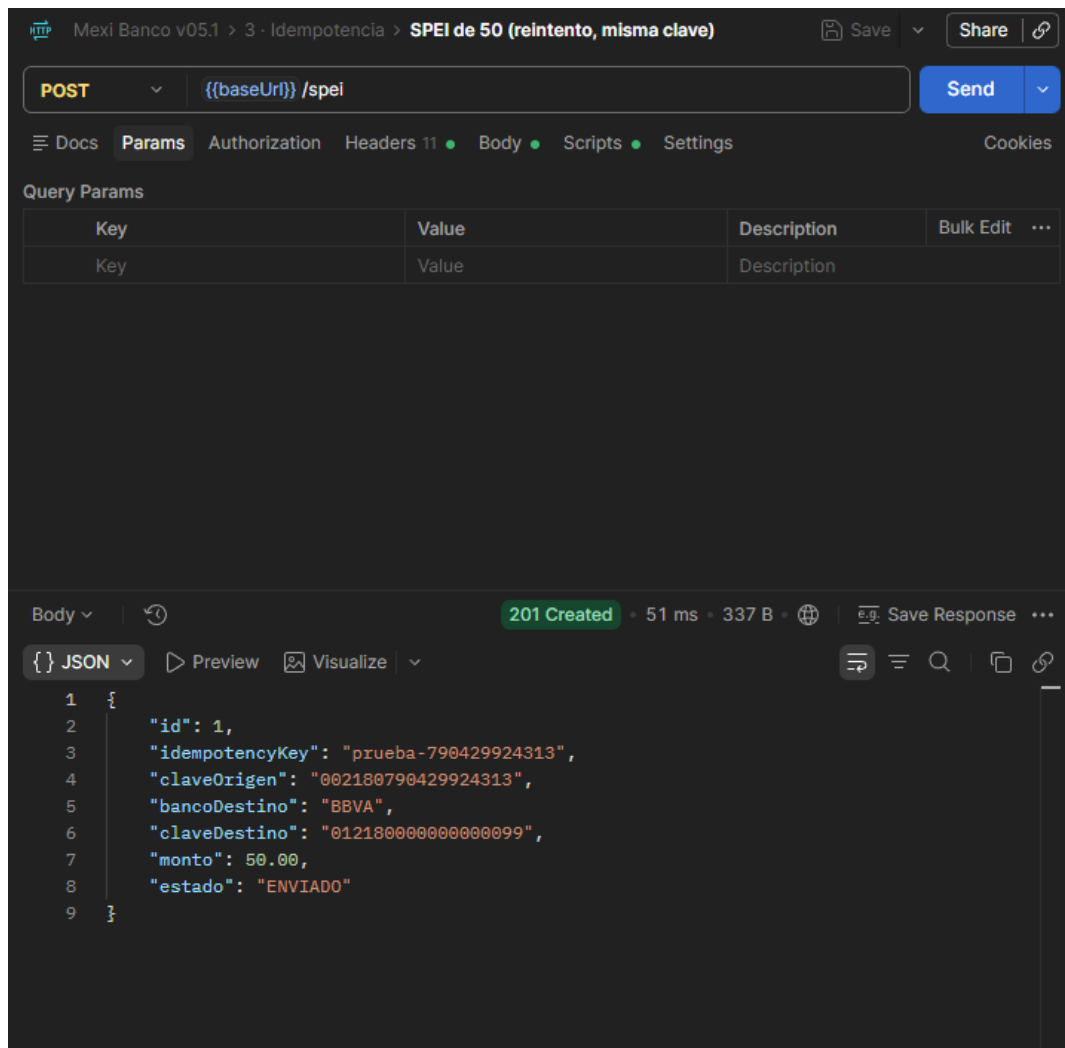


Figura 13: reintento de SPEI con la misma clave de idempotencia.

Paso 3.3. Idempotencia. Al consultar la cuenta origen, el saldo fue 750. Esto demostró que el descuento de 50 se hizo una sola vez.

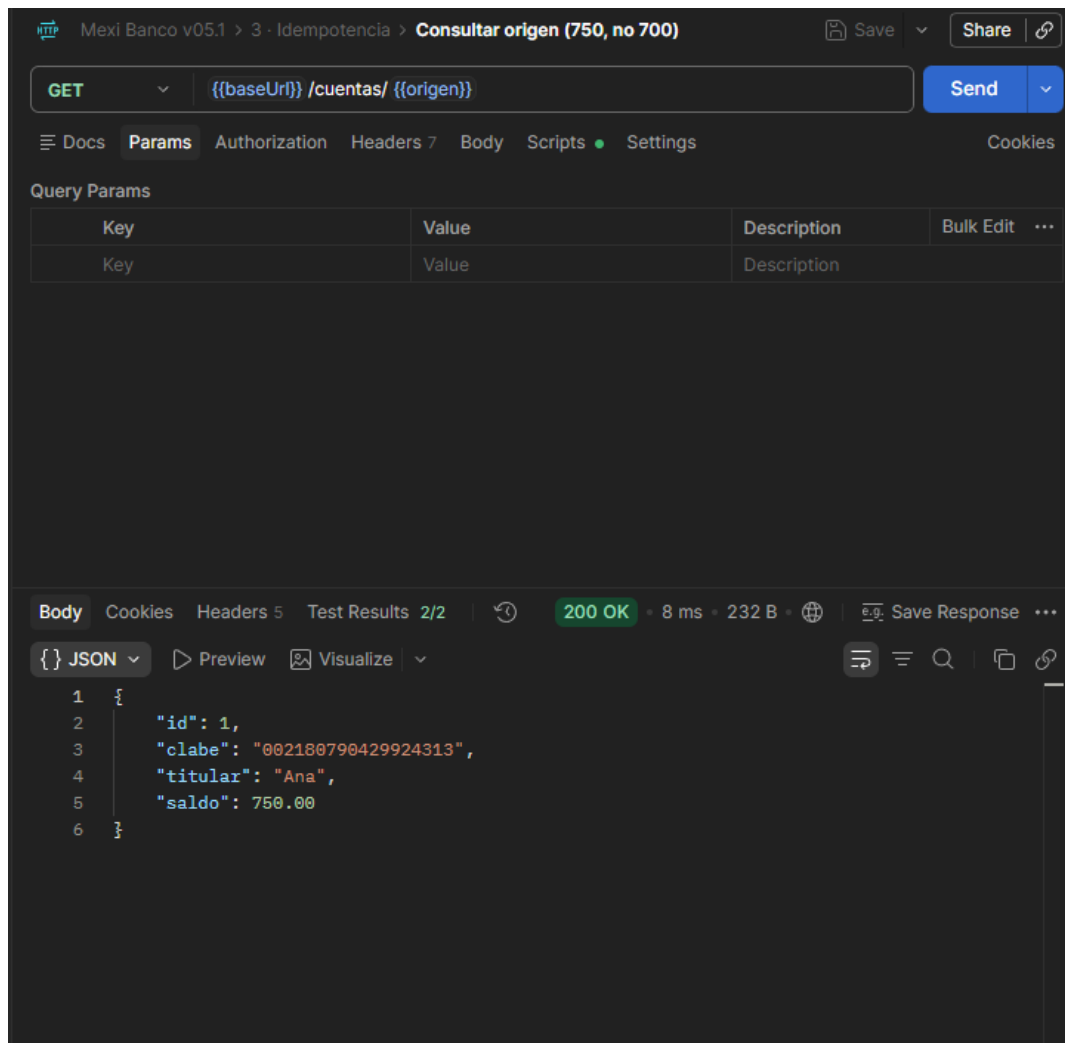


Figura 14: comprobación del saldo de la cuenta origen después del SPEI.

Paso 3.4. El monolito por dentro. Se consultó el estado de cuenta de Ana. La respuesta mostró el depósito inicial, la transferencia y el SPEI, junto con el saldo resultante de cada movimiento.

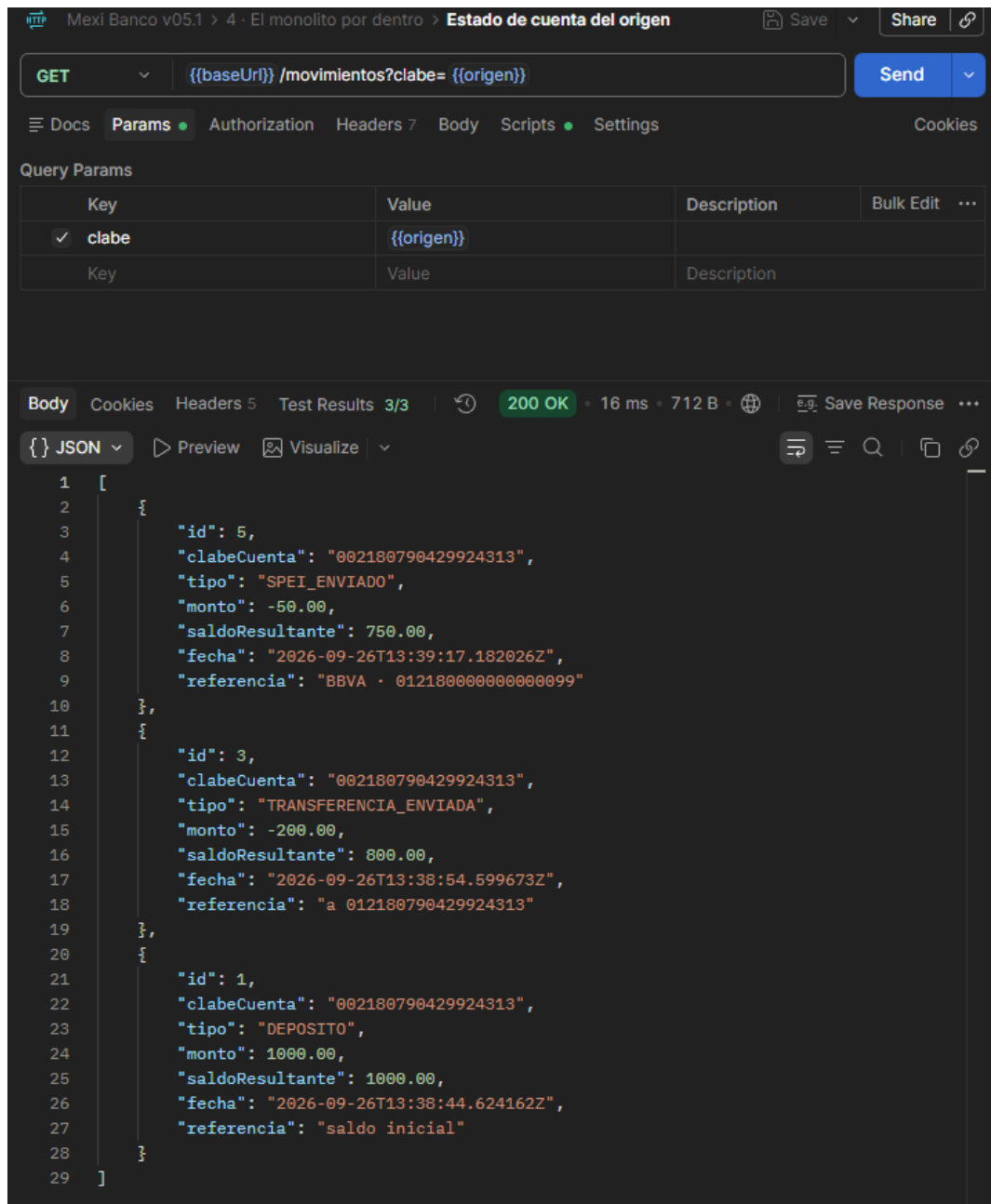


Figura 15: consulta del estado de cuenta de la cuenta origen.

Paso 3.4. El monolito por dentro. También se consultaron los avisos de la cuenta origen. Los mensajes confirmaron la transferencia de 200 y el SPEI de 50.

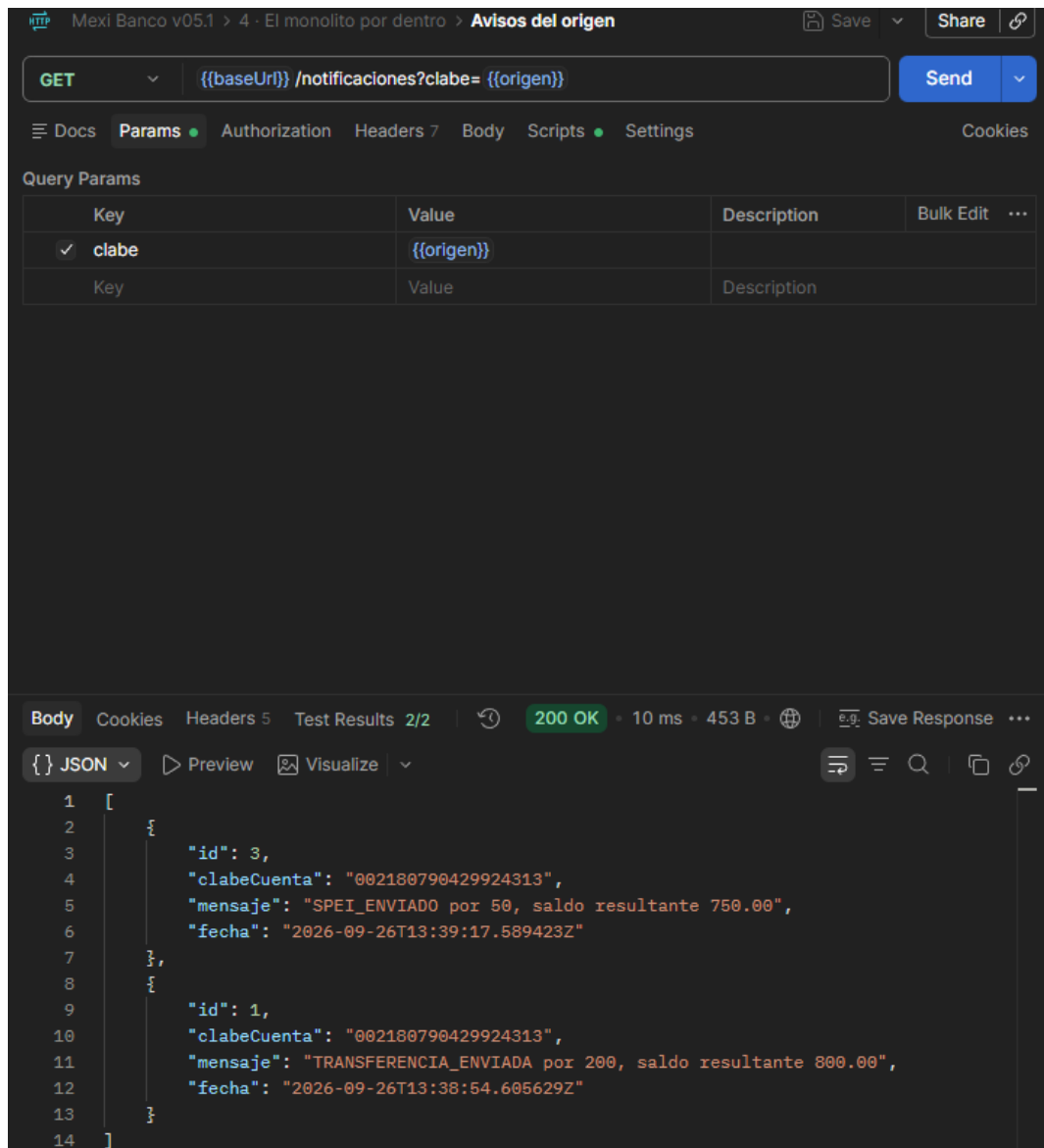


Figura 16: consulta de avisos generados para la cuenta origen.

Paso 3.5. Cuando algo sale mal. Se consultó una CLABE inexistente. La API respondió 404 Not Found, ya que no encontró la cuenta solicitada.

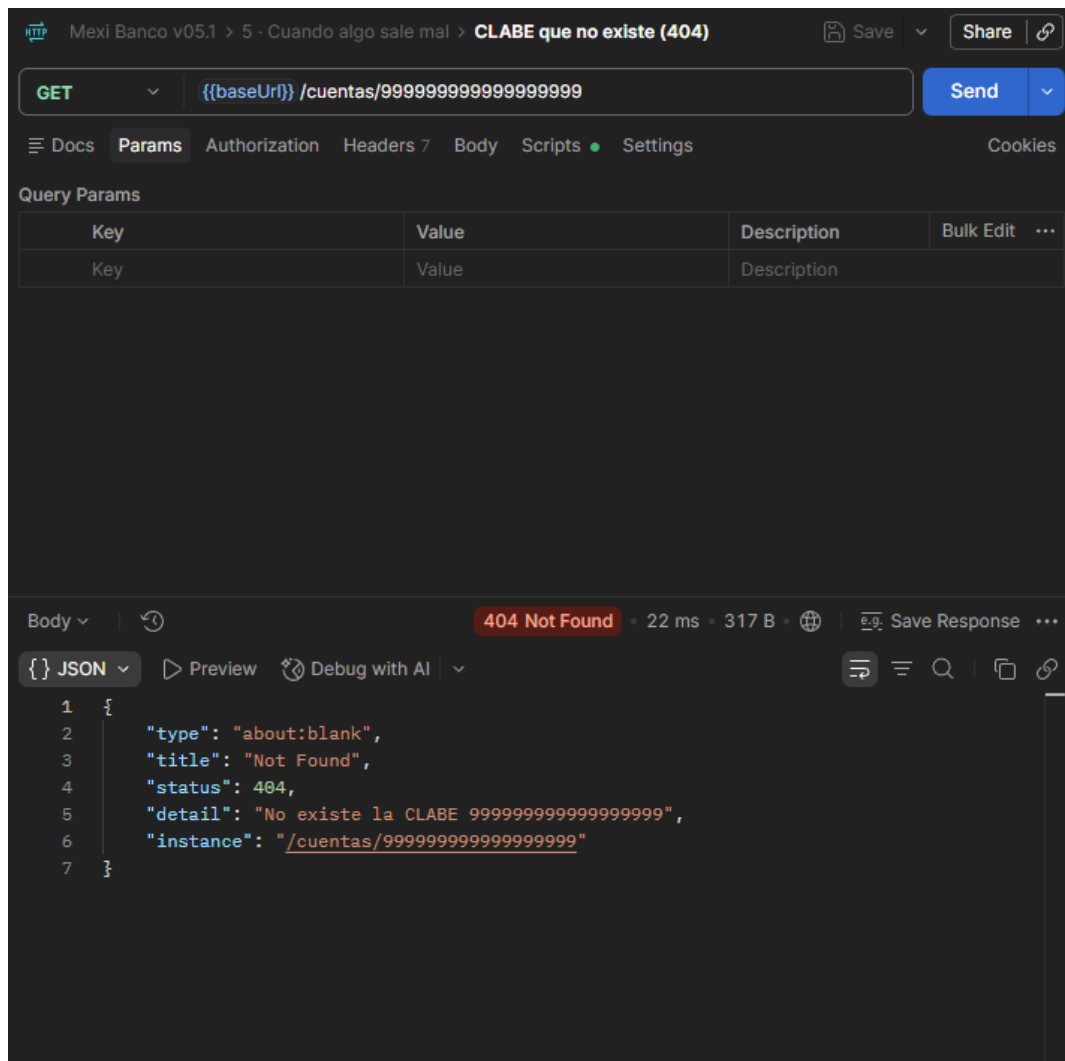


Figura 17: consulta de una CLABE inexistente.

Paso 3.5. Cuando algo sale mal. Se intentó crear una cuenta con una CLABE ya registrada. La respuesta fue 409 Conflict, porque no se permiten cuentas duplicadas.

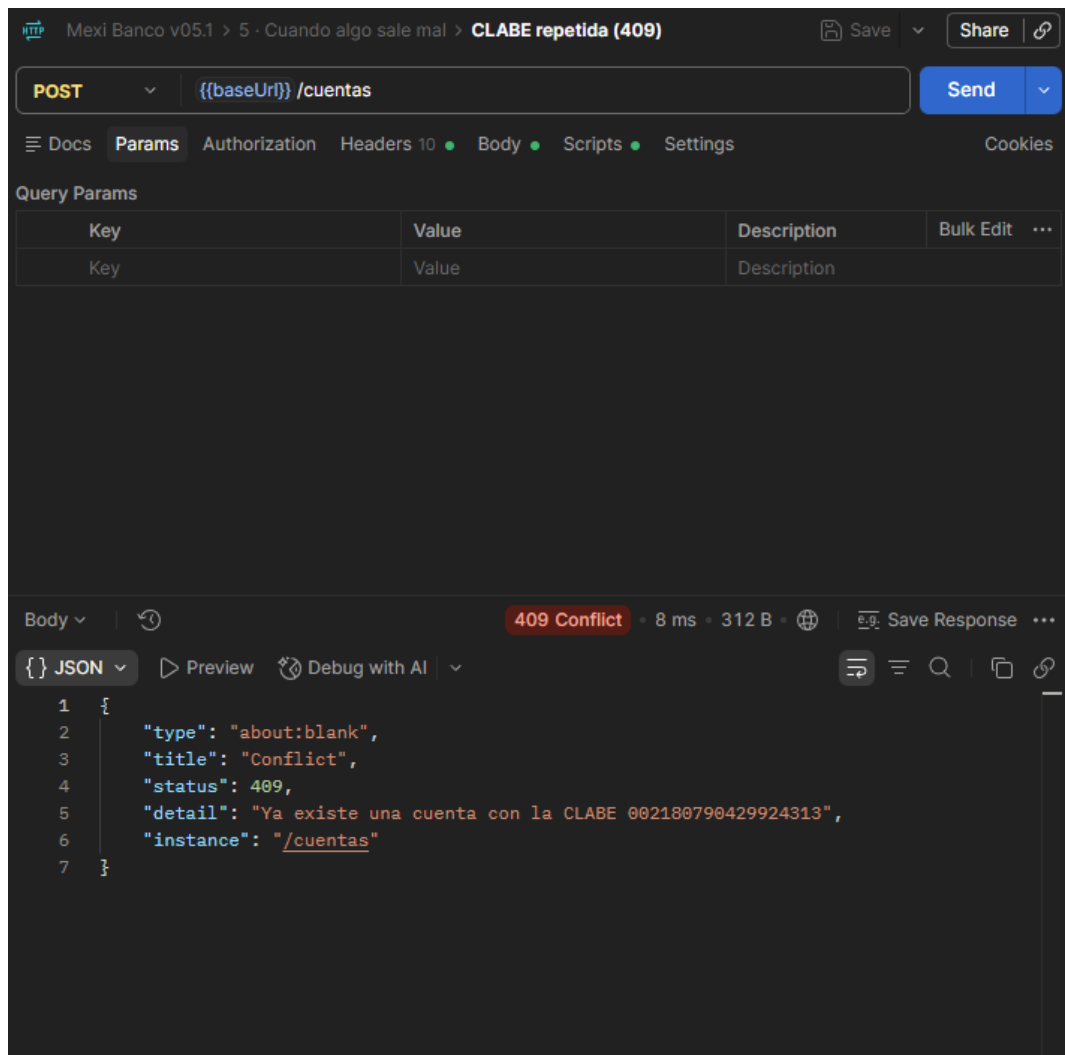


Figura 18: intento de registro con una CLABE repetida.

Paso 3.5. Cuando algo sale mal. Se envió una cuenta sin el campo CLABE. La solicitud no contenía la información necesaria y la API respondió 400 Bad Request.

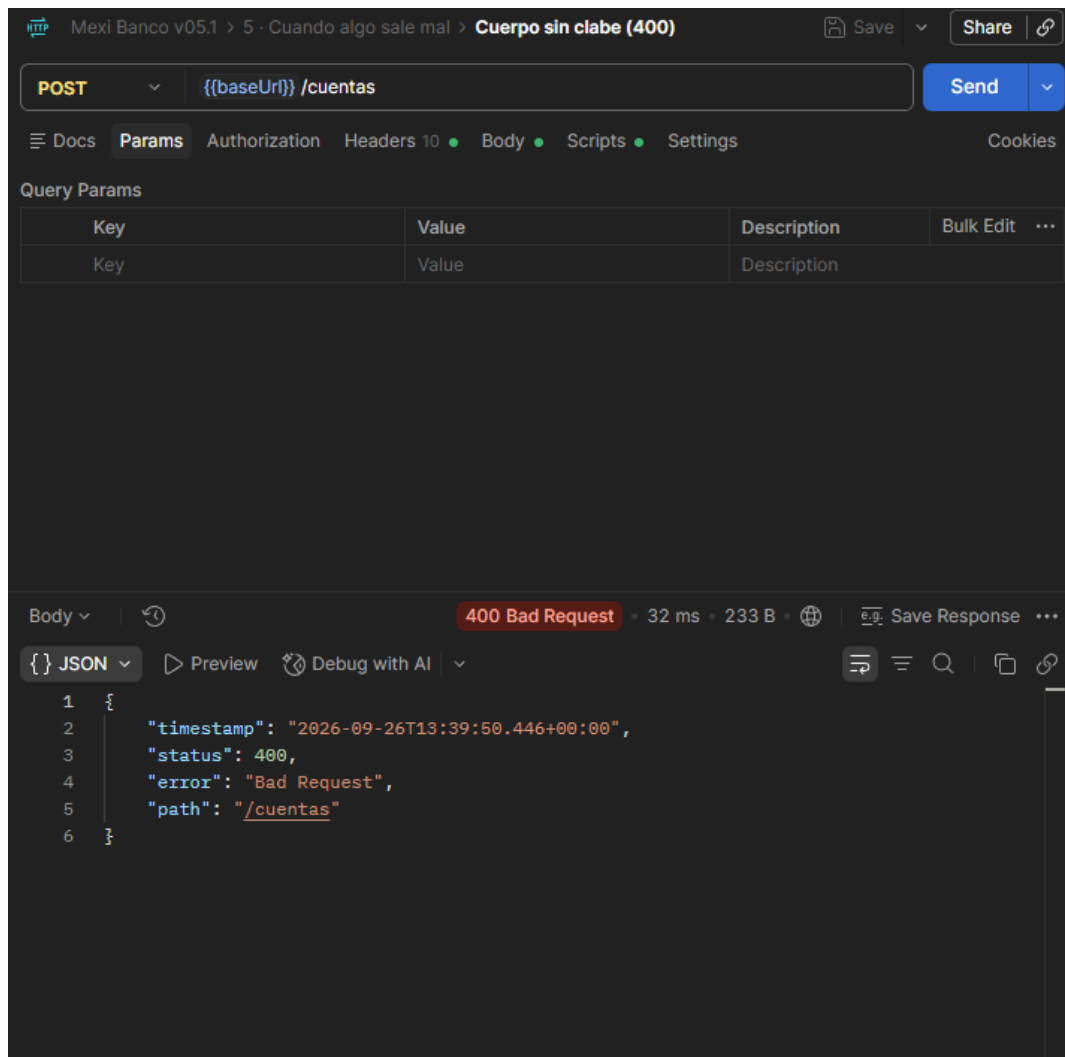


Figura 19: envío de una cuenta sin el campo CLABE.

Paso 3.5. Cuando algo sale mal. Se intentó transferir un monto mayor al saldo disponible. La API respondió 422 `Unprocessable Entity`, indicando saldo insuficiente.

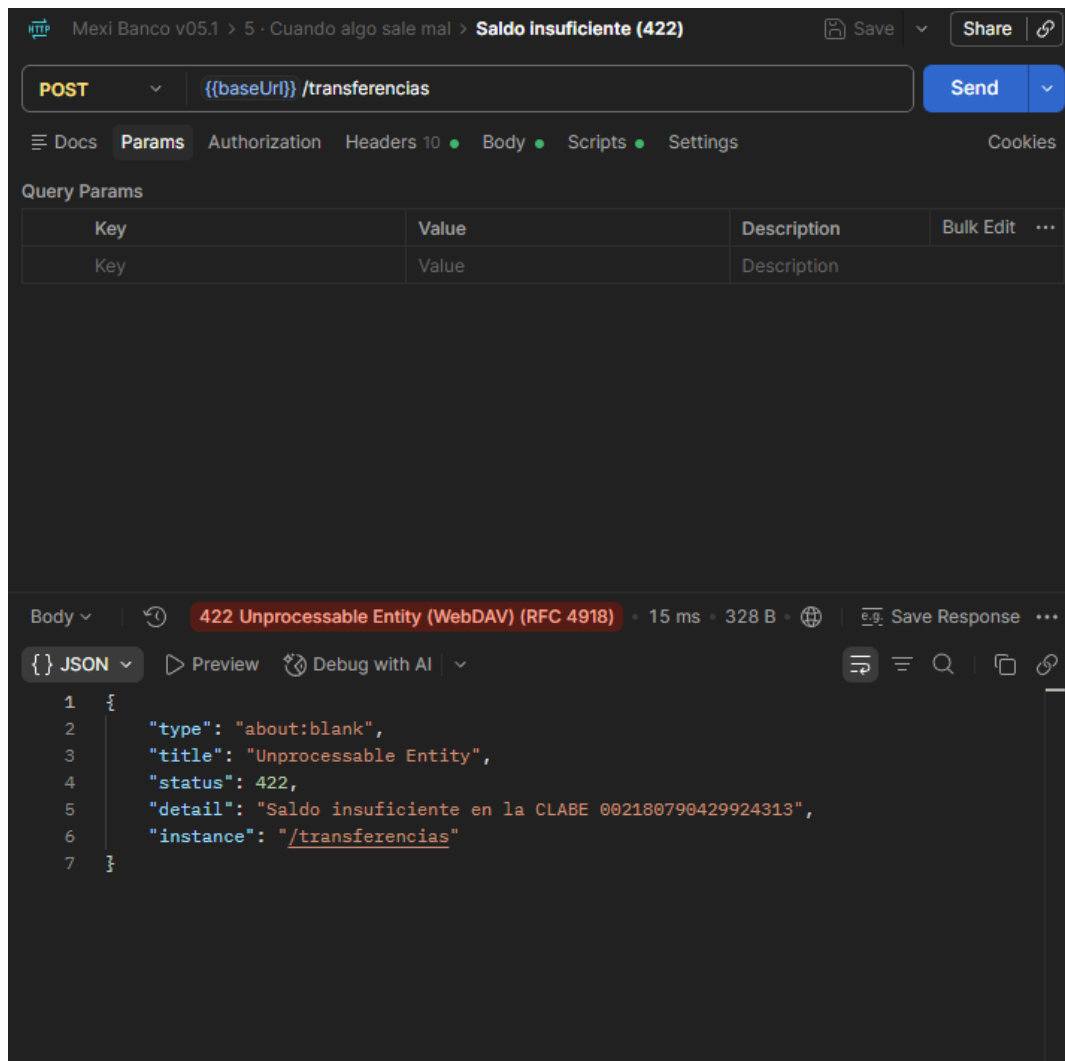


Figura 20: intento de transferencia con saldo insuficiente.

Paso 3.5. Cuando algo sale mal. Se envió una transferencia donde origen y destino eran la misma cuenta. La respuesta 422 confirmó que esta operación no es válida.

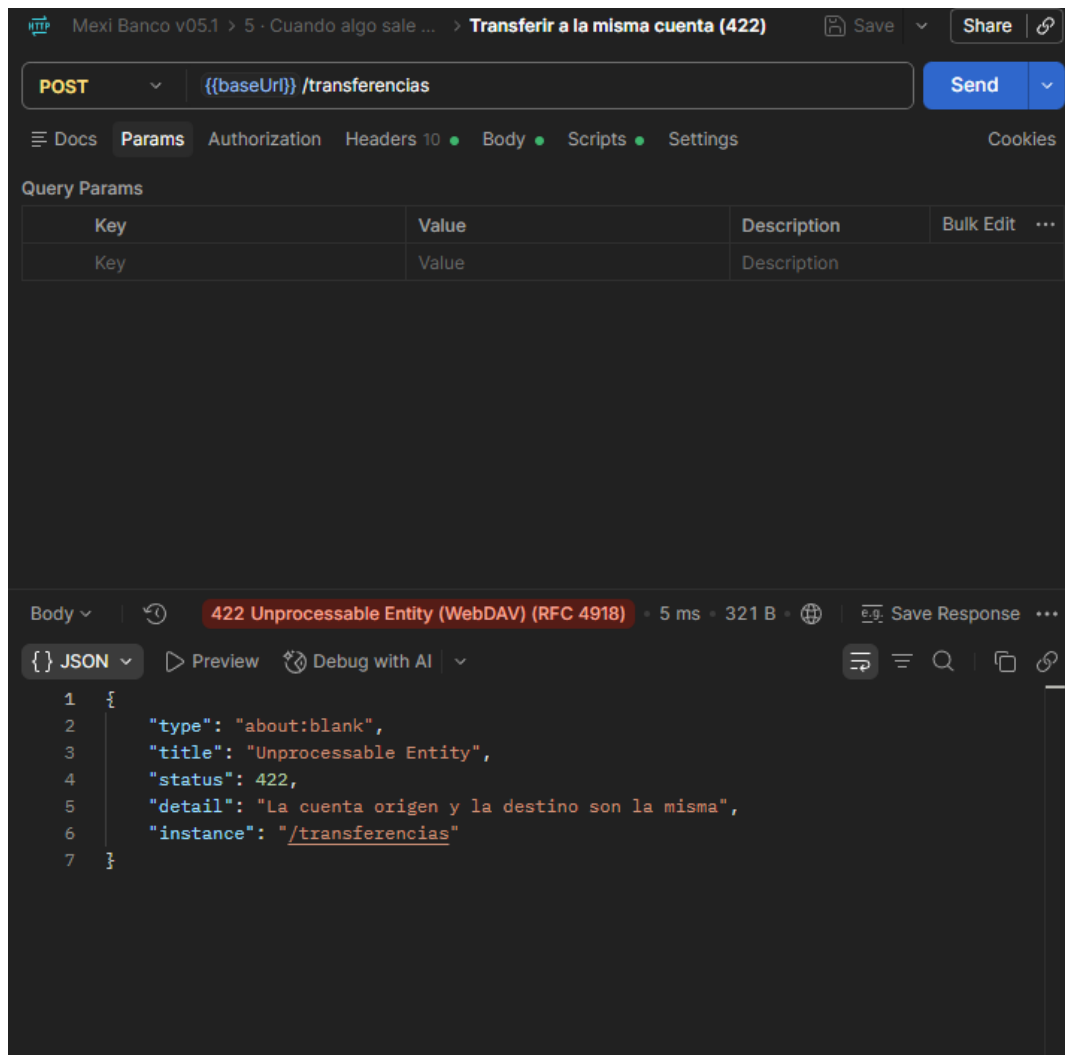


Figura 21: intento de transferir a la misma cuenta.

Paso 3.5. Cuando algo sale mal. Se intentó enviar un SPEI sin la cabecera *Idempotency-Key*. La API respondió 400 Bad Request, pues dicha clave es requerida.

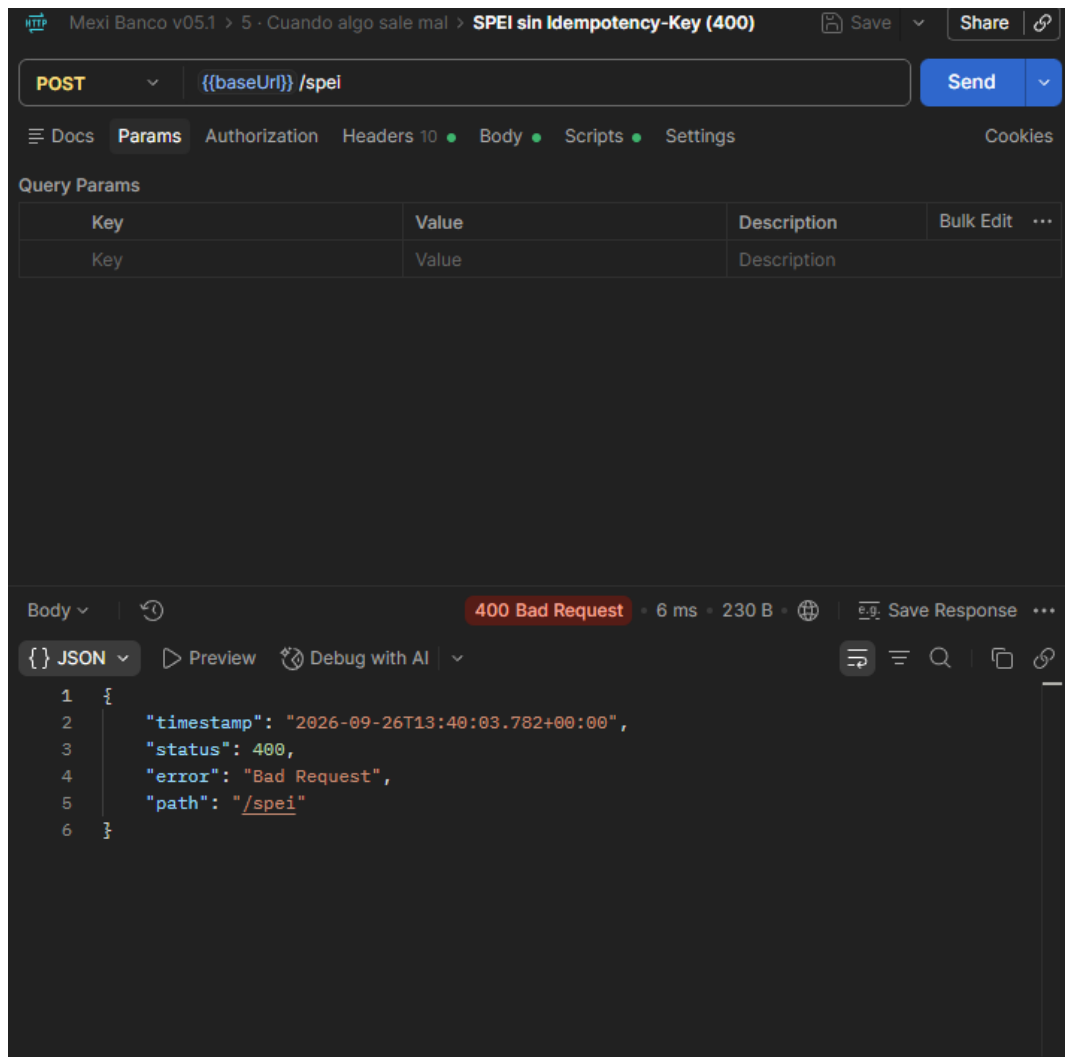


Figura 22: intento de enviar un SPEI sin clave de idempotencia.

Paso 3.5. Cuando algo sale mal. Después de las solicitudes inválidas se consultó de nuevo la cuenta de Ana. El saldo siguió en 750, de modo que los errores no generaron movimientos.

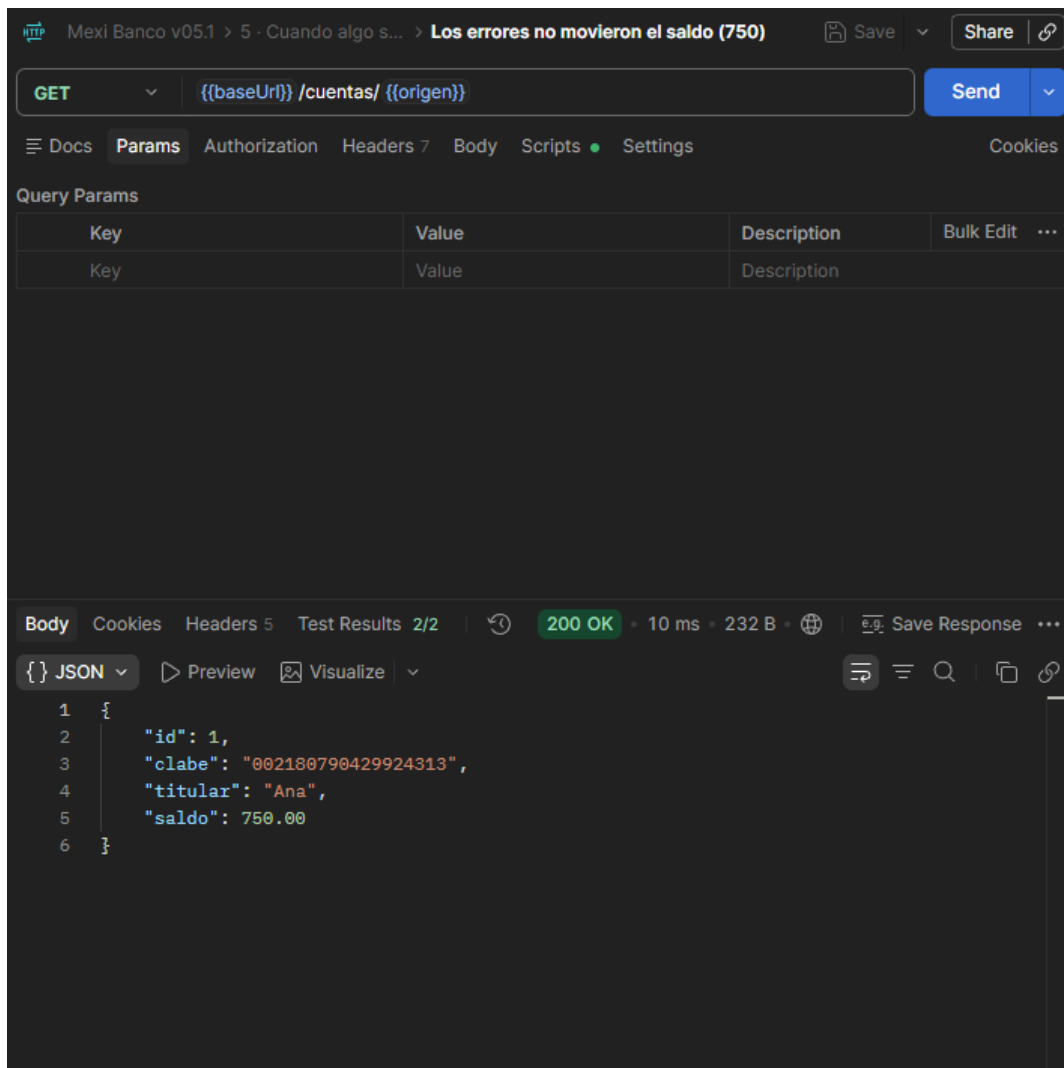


Figura 23: comprobación de que los errores no alteraron el saldo.

Paso 4. Apagado con y sin volumen. Primero se ejecutó `docker compose down` y después `docker compose up -d`. Los contenedores y la red se eliminaron, pero el volumen de PostgreSQL permaneció. **Paso 4.** En esta captura se observa el apagado sin la opción `-v`; por ello el volumen no se eliminó.

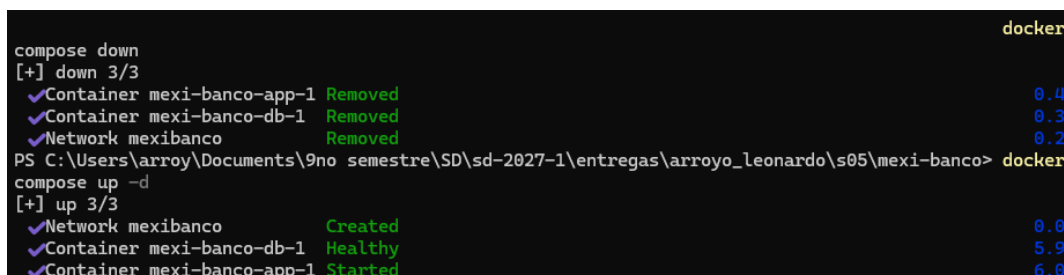


Figura 24: apagado y levantamiento de Compose sin eliminar el volumen.

Paso 4. Al consultar de nuevo la cuenta destino, esta continuó con saldo de 700. Esto demostró que los datos se conservaron en el volumen.

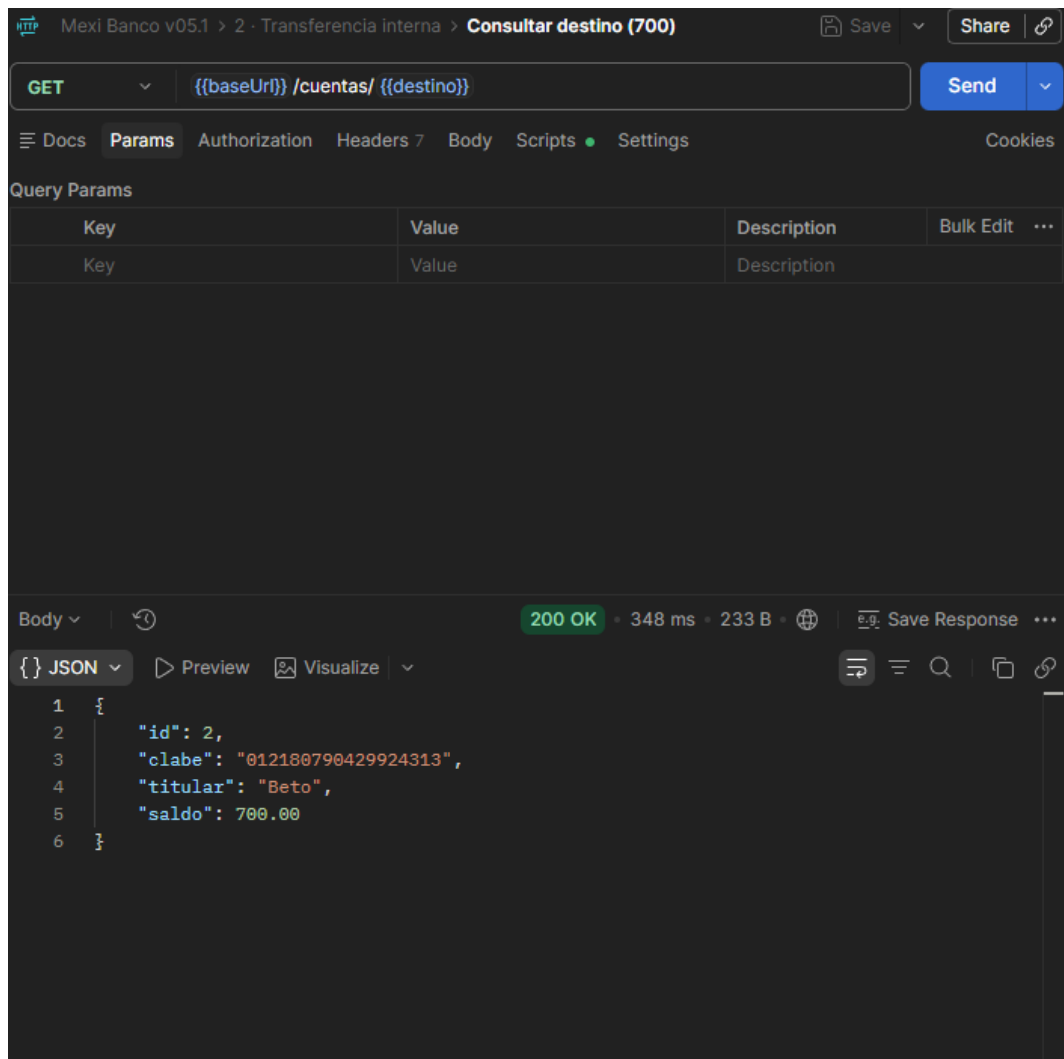


Figura 25: comprobación de la persistencia de datos sin borrar el volumen.

Paso 4. Luego se ejecutó `docker compose down -v`. La opción `-v` eliminó también el volumen `mexibanco_datos`.

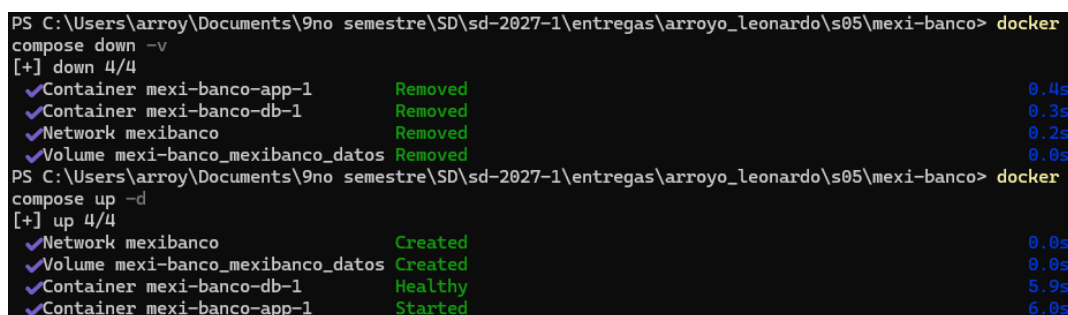


Figura 26: apagado de Compose eliminando también el volumen de datos.

Paso 4. Al levantar los servicios con un volumen nuevo, se consultó la cuenta destino y se obtuvo `404 Not Found`. Esto confirmó que los datos anteriores fueron eliminados.

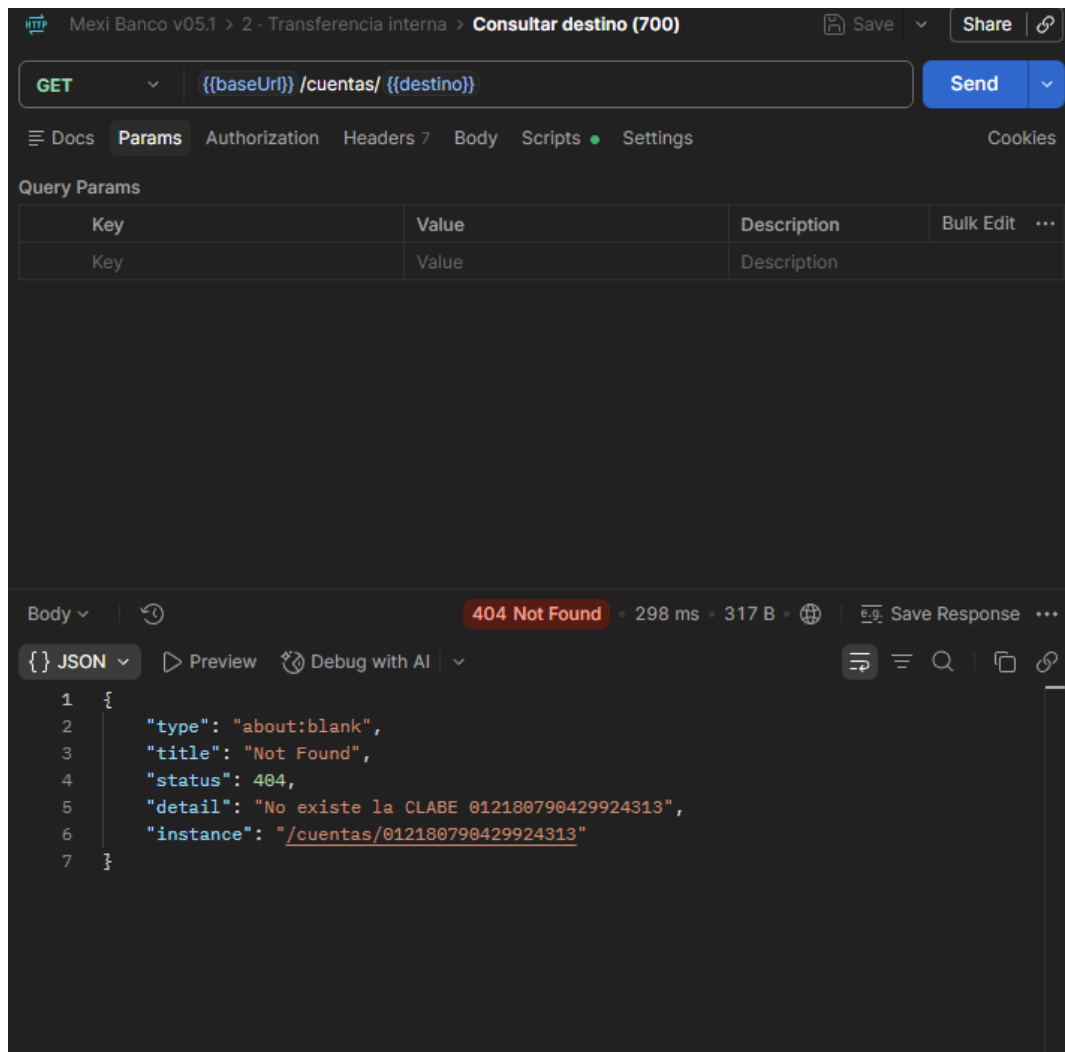


Figura 27: consulta posterior que confirma la eliminación de los datos.

Conclusión

La aplicación se levantó correctamente tras normalizar `mvnw`. Las pruebas confirmaron cuentas, transferencias, SPEI idempotente, movimientos, avisos y manejo de errores. También se comprobó que `docker compose down` conserva los datos y que `docker compose down -v` elimina el volumen que los guarda.

La práctica permitió comprobar la coordinación entre una aplicación y PostgreSQL con Docker Compose. Las pruebas de Postman mostraron tanto los casos correctos como los errores controlados. Finalmente, se observó de forma directa que el volumen es el elemento que mantiene la información persistente entre levantamientos de los contenedores.